

RAGMED: Sistema de Recuperación y Generación Aumentada para Diagnóstico de Enfermedades

Asignatura: Sistemas de Información No Estructurada (SINE)

Curso: 2025–2026

1. Introducción

Los sistemas de recuperación aumentada por generación (RAG) combinan tres componentes: un crawler que construye el corpus, un módulo de recuperación de información (IR) que localiza los fragmentos relevantes, y un LLM que genera la respuesta [1]. A diferencia de los LLMs puros, los sistemas RAG anclan las respuestas en el corpus, reduciendo alucinaciones sin necesidad de reentrenar el modelo.

Como punto de partida se proporcionó un sistema de referencia sobre información de Pokémon extraída de WikiDex [4,5]. Esta práctica extiende ese sistema al dominio médico: RAGMED recibe una lista de síntomas y sugiere posibles enfermedades apoyándose en un corpus de artículos de Wikipedia. El dominio médico es idóneo porque los artículos presentan estructura consistente (síntomas, causas, tratamiento), Wikipedia ofrece cobertura amplia y de libre acceso, y la aplicación tiene utilidad práctica como herramienta de orientación médica general.

Se desarrollaron tres componentes: (1) un crawler que descarga y estructura artículos de Wikipedia sobre enfermedades, (2) un módulo IR mejorado con recuperación híbrida BM25 + coseno, y (3) una evaluación manual con 10 preguntas. Las secciones 2–5 detallan cada componente; la sección 6 presenta las conclusiones.

2. Módulo Crawler

2.1 Descripción general y rol en el sistema RAG

Un crawler web es un programa automatizado que recorre páginas de internet siguiendo enlaces y extrae su contenido para construir un corpus de texto [1]. En el contexto de un sistema RAG, el crawler constituye la etapa de adquisición de datos: sin un corpus de calidad, los módulos de recuperación y generación no pueden ofrecer respuestas fundamentadas. La calidad del corpus —en términos de cobertura, limpieza y estructura— condiciona directamente la calidad de las respuestas finales del sistema.

Para RAGMED se implementó la clase `RAGMED_crawler` en el fichero `Sistema/ragmed_crawler.py`. El crawler cubre dos etapas: la descarga de la lista de enfermedades y la descarga del contenido individual de cada artículo.

En la **primera etapa**, el crawler recorre las páginas de índice de Wikipedia (letras A–Z) y extrae los nombres de enfermedades enlazadas, guardándolos en `disease_list.txt`. En la **segunda etapa**, para cada nombre de la lista, consulta la MediaWiki API para obtener el extracto de texto del artículo, lo segmenta en cuatro secciones y escribe el bloque resultante en `diseases.txt`.

2.2 Fuente de datos: Wikipedia mediante la API REST

La fuente de datos elegida es la Wikipedia en inglés, accedida a través de su API REST oficial (MediaWiki API) [2]. Se optó por Wikipedia porque reúne varias características que la hacen idónea para este dominio: es la mayor enciclopedia en línea de libre acceso, cubre miles de enfermedades con artículos de extensión y calidad variables pero generalmente aceptables, y su estructura de secciones es consistente entre artículos de la misma tipología.

La decisión de utilizar la API en lugar de scraping directo del HTML fue determinante para simplificar el pipeline. El parámetro `plaintext=true` de la API hace que el servidor devuelva el texto del artículo ya en formato plano, con los encabezados de

sección marcados mediante la convención == Nombre de sección ==. Esto elimina la necesidad de eliminar manualmente infoboxes, navboxes, superíndices de citas y otros elementos HTML ruidosos que sí habrían de tratarse en un scraping convencional. Además, el uso de la API respeta los términos de servicio de Wikimedia para usos no comerciales y académicos.

La URL del endpoint utilizado es la siguiente:

```
https://en.wikipedia.org/w/api.php?action=query&prop=extracts&explaintext=true
&titles={nombre_enfermedad}&format=json&redirects=1
```

El parámetro `redirects=1` permite que la API resuelva automáticamente los redireccionamientos de Wikipedia, de modo que nombres alternativos de una enfermedad apunten correctamente al artículo canónico.

Para construir la lista de enfermedades a descargar, el crawler recorre las páginas de índice alfabético de Wikipedia [3], desde la letra A hasta la Z:

```
https://en.wikipedia.org/wiki/List_of_diseases_(A)
...
https://en.wikipedia.org/wiki/List_of_diseases_(Z)
```

Estas páginas índice sí se accedieron mediante scraping HTML con BeautifulSoup [4], ya que la API no ofrece un endpoint equivalente para listar artículos de una categoría con la misma comodidad. De cada página índice se extraen los enlaces a artículos individuales (`<li>` con `<a href="/wiki/...">`) filtrando aquellos que contienen dos puntos en la ruta (que corresponden a páginas especiales como categorías o archivos) y los que no apuntan al espacio de nombres principal de Wikipedia.

2.3 Extracción y estructura del corpus

2.3.1 Selección de secciones

De cada artículo de enfermedad se extrajeron cuatro fragmentos:

- **Párrafo inicial (lead):** el texto antes del primer encabezado de sección, que constituye el resumen general de la enfermedad.
- **Signs and symptoms** (síntomas y signos): es la sección que describe las manifestaciones clínicas, directamente relevante para la tarea de diagnóstico a partir de síntomas.
- **Causes** (causas): explica la etiología de la enfermedad, lo que complementa la descripción clínica.
- **Treatment / Management / Prevention** (tratamiento): se busca en ese orden de prioridad, dado que distintos artículos emplean denominaciones diferentes para la misma sección temática.

Estas tres secciones se seleccionaron porque responden directamente a la pregunta que el sistema debe resolver: dado un conjunto de síntomas, ¿qué enfermedad puede ser y cómo se trata? El párrafo inicial aporta el contexto general, los síntomas permiten la recuperación por similitud con la consulta del usuario, las causas enriquecen la explicación, y el tratamiento completa la información clínicamente relevante.

La extracción de secciones a partir del texto plano devuelto por la API se realizó mediante una expresión regular que divide el texto en el patrón == Sección ==:

```
_H2_PATTERN = re.compile(r"^== (.+?) ==$", re.MULTILINE)
```

La función `_parse_sections` divide el texto completo en un par (párrafo inicial, diccionario de secciones), y `_find_section` recorre una lista de nombres candidatos en orden de prioridad hasta encontrar una coincidencia.

2.3.2 Formato de salida

El corpus final se almacena en un fichero `diseases.txt` donde cada enfermedad ocupa un bloque con el siguiente formato:

```
Nombre de la enfermedad
=====
Lead: [descripción general]
```

```

Signs and symptoms
=====
[texto]

Causes
=====
[texto]

Treatment
=====
[texto]

```

Cuando una sección no está disponible en el artículo de Wikipedia, se escribe el literal (No information available.) para que el bloque mantenga siempre la misma estructura y sea fácilmente parseable por el módulo de recuperación. Además de este fichero consolidado, el crawler genera ficheros intermedios por enfermedad (diseases/{nombre}.txt para el extracto bruto y diseases/{nombre}_clean.txt para el extracto estructurado), lo que facilita la depuración y la reanudación de ejecuciones interrumpidas.

El siguiente fragmento ilustra el formato resultante para una enfermedad bien documentada:

```

Abdominal aortic aneurysm
=====
Lead: Abdominal aortic aneurysm (AAA) is a localized enlargement of the
abdominal aorta such that the diameter is greater than 3 cm [...]

Signs and symptoms
=====
The vast majority of aneurysms are asymptomatic. However, as the aorta
expands, the aneurysm may become painful [...]

Causes
=====
Tobacco smoking: More than 90% of people who develop an AAA have smoked [...]

Treatment
=====
Treatment options are conservative management, surveillance, and repair [...]

```

2.3.3 Problemas encontrados y soluciones

Durante el desarrollo del crawler se identificaron tres categorías de problemas:

Ruido en las páginas índice. Las páginas `List_of_diseases_(X)` contienen, además de los enlaces a artículos de enfermedades, elementos de navegación como navboxes, tablas de contenidos y hatnotes. Al iterar sobre todos los elementos `<a>` del contenido principal, algunos de estos enlaces de navegación se colaban en la lista. La solución adoptada fue eliminar explícitamente los nodos de ruido (navbox, toc, mw-references-wrap, hatnote) antes de recorrer los `<li>`, y además filtrar cualquier href que contuviera dos puntos o que no comenzara por `/wiki/`.

Limitación de tasa de peticiones (HTTP 429). La Wikipedia impone límites de velocidad a los clientes que realizan peticiones rápidas. Cuando el servidor devuelve un código 429 (*Too Many Requests*) o 503, el crawler implementa una estrategia de reintento con espera exponencial: 2 segundos tras el primer intento fallido, 4 tras el segundo y 8 tras el tercero (`wait = 2 ** attempt`). Adicionalmente, entre cada petición a la API se introduce una espera fija de 1,5 segundos. La cabecera `User-Agent` se configura con el valor `RAGMED-Crawler/1.0 (university homework)` para identificar el cliente de forma transparente, tal como recomienda la política de uso de la API de Wikimedia [2].

Inconsistencia en los nombres de sección. Wikipedia no normaliza de forma estricta los títulos de las secciones entre artículos. Por ejemplo, mientras que Influenza usa la sección `Treatment`, Diabetes mellitus emplea `Management`. El crawler maneja esta variabilidad buscando los nombres candidatos en orden de prioridad: `["Treatment",`

"Management", "Prevention"]. Esta lista se define como constante en el módulo y puede ampliarse fácilmente si se identifican otras denominaciones en el corpus.

Toda la actividad del crawler se registra mediante el módulo estándar `logging` de Python, con nivel `INFO` para el progreso normal y `WARNING/ERROR` para situaciones de fallo. Se evitó el uso de sentencias `print` directas para facilitar la integración futura del módulo con sistemas de orquestación que capturen logs estructurados.

2.3.4 Resultados de la ejecución

Se ejecutó el crawler sobre el índice alfabético completo de Wikipedia (letras A–Z). El proceso identificó **5.390 nombres de enfermedades** en las páginas de índice y descargó con éxito el artículo de **4.312 de ellas**, generando un corpus consolidado de 146.806 líneas en `diseases.txt`. Las 1.078 entradas restantes corresponden principalmente a artículos vacíos, redirecciones a páginas de desambiguación o síndromes muy raros con escasa documentación en Wikipedia, donde la API devuelve un extracto nulo o la página está marcada como `missing`.

La calidad varía notablemente: enfermedades bien documentadas disponen de las cuatro secciones completas, mientras que síndromes raros solo tienen párrafo inicial y quedan con los campos de síntomas y causas marcados como `(No information available.)`.

3. Módulo de Recuperación de Información

El módulo de recuperación de información es el componente central de todo sistema RAG: determina qué fragmentos del corpus se proporcionan al modelo de lenguaje como contexto y, por tanto, condiciona directamente la calidad de la respuesta generada. En RAGMED, este módulo está implementado en la clase `RAGMED_rag` del fichero `Sistema/ragmed_rag.py` y extiende el sistema de referencia con tres mejoras sustanciales: una estrategia de fragmentación semántica, un conjunto ampliado de funciones de similitud, y un modo de recuperación híbrida que combina métodos dispersos y densos.

3.1 Sistema base: similitud coseno sobre embeddings densos

La clase de referencia `SINE_rag` implementa recuperación densa mediante embeddings `bge-base-en-v1.5-gguf` (768 dimensiones, L2-normalizados) con similitud coseno, que se reduce al producto escalar [1, §7.1.2]. Captura equivalencias semánticas automáticamente (“shortness of breath” → “dyspnea”), pero no discrimina por frecuencia del término y puede penalizar términos clínicos raros poco frecuentes en el corpus de preentrenamiento.

3.2 Mejora implementada: recuperación híbrida BM25 + embedding coseno

La principal mejora es la introducción de recuperación híbrida BM25 + coseno. Los dos componentes son complementarios: BM25 (Okapi BM25) sobresale cuando la consulta contiene terminología clínica exacta (“haemoptysis”, “petechiae”) gracias a su IDF y saturación de frecuencia de término; los embeddings densos capturan equivalencias semánticas cuando el usuario emplea lenguaje coloquial (“shortness of breath” → “dyspnea”, “yellowing of the skin” → “jaundice”) [1, §7.4, p. 267].

La función de puntuación híbrida implementada en `RAGMED_rag.retrieve_function` combina ambas señales mediante una suma ponderada:

$$\text{score_hybrid}(Q, D) = \alpha \cdot \text{coseno_emb}(Q, D) + (1 - \alpha) \cdot \text{BM25_norm}(Q, D)$$

donde $\alpha \in [0, 1]$ es el parámetro de mezcla (valor por defecto: 0,5), `coseno_emb` es la similitud coseno entre los embeddings de la consulta y el fragmento (rango [0, 1] para embeddings no negativos), y `BM25_norm` es la puntuación BM25 normalizada por el máximo del lote para llevarla al intervalo [0, 1]:

$$\text{BM25_norm}(Q, D) = \frac{\text{BM25}(Q, D)}{\max_{D' \in \mathcal{C}} \text{BM25}(Q, D') + \epsilon}$$

con $\varepsilon = 10^{-8}$ para evitar la división por cero. La normalización por máximo preserva el ordenamiento relativo de las puntuaciones BM25 dentro del lote y las hace directamente comparables a las similitudes coseno, que ya están acotadas en $[0, 1]$ [1, §7.4, p. 267].

La implementación utiliza la variante `BM25Okapi` de la biblioteca `rank_bm25` [6], con parámetros por defecto `k1=1.5`, `b=0.75`. La tokenización del corpus y de la consulta se realiza de forma consistente: conversión a minúsculas y separación por espacios. El índice BM25 se construye una sola vez en el método `load_dataset`, junto con el proceso de embedding del corpus, y queda disponible en memoria para todas las consultas posteriores.

3.3 Funciones de similitud implementadas

`RAGMED_rag` expone cuatro funciones de similitud individuales más el modo híbrido, seleccionables mediante el parámetro `similarity_fn` del constructor o la opción `--similarity-fn` de la interfaz de línea de comandos. La Tabla 1 resume sus características y su adecuación al dominio médico.

Función	Tipo	Descripción breve	Idoneidad para RAGMED
Coseno (denso)	Densa	Ángulo entre vectores de embedding	Buena (maneja sinónimos)
BM25 (Okapi)	Dispersa	TF saturado + IDF + normaliz. longitud	Buena (términos exactos)
Euclidiana	Densa	$1/(1+\text{distancia L2})$	Moderada
Jaccard	Dispersa	Solapamiento de conjuntos de tokens	Débil
Híbrida (BM25+coseno)	Mixta	Suma ponderada	Óptima

Tabla 1: Funciones de similitud implementadas en `RAGMED_rag` y su adecuación al dominio médico.

La euclidiana captura la misma señal que el coseno con vectores L2-normalizados; la Jaccard opera sobre conjuntos de tokens sin ponderación y es la opción más débil al dar el mismo peso a “fever” (frecuente) que a “haemoptysis” (discriminativo) [1, §7.1.1]. Ambas se incluyen con fines comparativos.

3.4 Estrategia de chunking y parámetros de configuración

A diferencia del sistema de referencia, que trata cada línea del fichero de corpus como un fragmento independiente, `RAGMED_rag` implementa una estrategia de fragmentación a nivel de sección. El método `_parse_chunks` analiza el fichero `diseases.txt` e identifica los bloques de enfermedad por su patrón de encabezado (`{Nombre}\n{'='*n}\n`); a continuación, `_extract_disease_chunks` genera hasta cuatro fragmentos por enfermedad, correspondientes a las secciones Lead, Signs and symptoms, Causes y Treatment. Cada fragmento se prefija con el nombre de la enfermedad siguiendo el patrón `{Nombre de la enfermedad} – {Sección}: {texto}`, lo que garantiza que el modelo de lenguaje disponga siempre de contexto para atribuir los síntomas a su enfermedad correspondiente.

La fragmentación por sección mejora la precisión: el fragmento de “Signs and symptoms” contiene solo el texto relevante para la consulta, sin ruido etiológico o terapéutico. Los fragmentos vacíos o con `(No information available.)` se descartan para no consumir posiciones en el ranking.

Los parámetros principales del sistema se recogen en la Tabla 2:

Parámetro	Valor por defecto	Descripción
<code>similarity_fn</code>	<code>hybrid</code>	Función de recuperación activa
<code>alpha</code>	<code>0,5</code>	Peso del coseno en modo híbrido

Parámetro	Valor por defecto	Descripción
top_n	5	Número de fragmentos recuperados
Modelo de embedding	bge-base-en-v1.5-gguf	Embeddings de 768 dimensiones
Modelo de lenguaje	Llama-3.2-1B-Instruct-GGUF	Generación de respuestas

Tabla 2: Parámetros de configuración de RAGMED_rag.

El parámetro `alpha` y el número de resultados `top_n` pueden ajustarse desde la interfaz de línea de comandos mediante las opciones `--alpha` y `--top-n` respectivamente, lo que permite explorar el espacio de configuraciones sin modificar el código fuente.

4. Evaluación

4.1 Metodología de evaluación

La evaluación del sistema RAGMED se realizó mediante un protocolo de evaluación manual compuesto por diez preguntas de ejemplo con respuesta de referencia (*ground truth*) definida a priori. Cada pregunta está formulada en inglés, en primera persona y desde la perspectiva de un paciente que describe sus síntomas, lo que reproduce el caso de uso previsto del sistema. El idioma inglés se eligió para maximizar la calidad de la recuperación, ya que el corpus está íntegramente en ese idioma.

Para cada pregunta, se identificó manualmente la enfermedad esperada y se localizó el pasaje del corpus (`diseases.txt`) que avala la respuesta correcta. Este procedimiento garantiza que la respuesta de referencia procede exclusivamente del conocimiento contenido en el corpus, en coherencia con el principio de grounding del paradigma RAG.

El sistema se evaluó con la configuración por defecto: función de recuperación híbrida (BM25 + coseno, $\alpha = 0,5$) y cinco fragmentos recuperados por consulta. Cada respuesta generada se comparó con la respuesta de referencia y se puntuó con tres métricas independientes, siguiendo la rúbrica de la Tabla 3:

Métrica	0	0,5	1
Precisión	Respuesta incorrecta o irrelevante	Identificación parcialmente correcta	Identificación correcta con síntomas clave
Cobertura	No cubre los síntomas de la consulta	Cubre parcialmente los síntomas relevantes	Cubre todos los síntomas clave del corpus
Veracidad	Contiene afirmaciones falsas (alucinaciones)	Información dudosa o imprecisa	Sin información inventada

Tabla 3: Rúbrica de puntuación para la evaluación manual.

La puntuación máxima posible es 30 puntos (10 preguntas $\times$ 3 métricas $\times$ 1 punto).

4.2 Preguntas de evaluación

Las diez preguntas cubren categorías diagnósticas diversas, garantizando que el sistema sea evaluado en distintos dominios médicos. La Tabla 4 resume las preguntas y la enfermedad esperada en cada caso. El documento completo con las preguntas, los pasajes de referencia y los criterios de corrección se encuentra en `Evaluación/preguntas_ground_truth.md`.

#	Consulta (resumen)	Enfermedad esperada	Categoría
Q1	Mocos, dolor de garganta, estornudos, tos leve	Common cold	Viral/infecciosa

#	Consulta (resumen)	Enfermedad esperada	Categoría
Q2	Fiebre súbita, dolores musculares, escalofríos, tos seca	Influenza	Viral/infecciosa
Q3	Sibilancias, disnea y opresión torácica nocturna	Asthma	Respiratoria crónica
Q4	Cefalea pulsátil unilateral, náuseas, fotofobia y fonofobia	Migraine	Neurológica
Q5	Dolor súbito, inflamación y enrojecimiento en el dedo gordo del pie	Gout	Metabólica/reumática
Q6	Fatiga extrema, aumento de peso, intolerancia al frío, estreñimiento	Hypothyroidism	Endocrina
Q7	Fiebre, dolor torácico pleurítico, tos productiva, disnea	Pneumonia	Respiratoria/infecciosa
Q8	Articulaciones inflamadas, rigidez matutina >1 h, afectación simétrica	Rheumatoid arthritis	Autoinmune
Q9	Fiebre cíclica, escalofríos, cefalea y náuseas tras viaje a África	Malaria	Parasitaria/tropical
Q10	Tos crónica con sangre, sudores nocturnos, pérdida de peso	Tuberculosis	Bacteriana/crónica

Tabla 4: Preguntas de evaluación y enfermedades esperadas.

4.3 Resultados

Los resultados obtenidos tras ejecutar el script de evaluación (`Sistema/ragmed_eval.py`) con la configuración por defecto se recogen en la Tabla 5. Las respuestas completas del sistema se encuentran en `Evaluación/respuestas_sistema.md`; las puntuaciones detalladas y observaciones por pregunta, en `Evaluación/evaluacion.md`.

#	Enfermedad	Precisión	Cobertura	Veracidad	Total
Q1	Common cold	1	1	1	3/3
Q2	Influenza	0,5	0,5	0,5	1,5/3
Q3	Asthma	1	1	1	3/3
Q4	Migraine	1	0,5	0,5	2/3
Q5	Gout	1	0,5	0,5	2/3
Q6	Hypothyroidism	0,5	0	0,5	1/3
Q7	Pneumonia	1	1	1	3/3
Q8	Rheumatoid arthritis	0,5	0,5	1	2/3
Q9	Malaria	0	0	0	0/3
Q10	Tuberculosis	0	0	0,5	0,5/3
Total		6,5/10	5/10	6/10	17,5/30
%		65 %	50 %	60 %	58,3 %

Tabla 5: Resultados de la evaluación manual.

4.4 Análisis

La evaluación arrojó una puntuación global de **17,5/30 (58,3 %)**, con una distribución claramente bimodal: tres preguntas obtuvieron la máxima puntuación (Q1, Q3, Q7) y tres obtuvieron puntuación casi nula (Q6, Q9, Q10).

Fortalezas del sistema. El sistema funciona de forma óptima cuando los síntomas de la consulta coinciden léxicamente con el corpus. En Q1, Q3 y Q7, el fragmento correcto alcanzó la primera posición con puntuaciones >0,85, donde el componente BM25 refuerza términos exactos (“wheezing”, “chest tightness”) y el coseno captura coherencia semántica.

Fallos de recuperación. Se identificaron dos patrones de fallo:

1. *Brecha léxica.* En la pregunta Q6 (hypothyroidism), la consulta emplea lenguaje coloquial (“feeling cold all the time”) que no coincide con la terminología clínica del corpus (“poor ability to tolerate cold”). Esta discrepancia impidió que ninguno de los cinco fragmentos recuperados perteneciera al artículo de hypothyroidism. El componente semántico del embedding debería mitigar este tipo de brecha, pero la magnitud del corpus (~11 000 fragmentos) y la presencia de muchas enfermedades con síntomas genéricos de fatiga o malestar general parece saturar el espacio semántico.
2. *Síntomas compartidos.* En Q9 (malaria) y Q10 (tuberculosis), los síntomas descritos —fiebre cíclica, escalofríos y tos con hemoptisis, sudores nocturnos y pérdida de peso— son hallazgos presentes en numerosas enfermedades. Sin el nombre de la enfermedad en la consulta, el ranking no favorece a los artículos específicos de malaria y tuberculosis frente a enfermedades con perfiles sintomáticos solapados.

Comportamiento de seguridad del modelo. En Q9, el modelo de lenguaje rechazó responder con el mensaje “I can’t provide information about diseases”, a pesar del system prompt configurado como asistente de información médica. Este comportamiento de autorestricción del modelo base reduce la utilidad del sistema para enfermedades tropicales y merece atención en una mejora futura.

Alucinación puntual. En Q5 (gout), el modelo cometió el error factual “allopurinol (Zyrtec)”, confundiendo el nombre comercial de la cetirizina (antihistamínico) con el del alopurinol. Este tipo de error muestra que el modelo de 1B parámetros mezcla en ocasiones conocimiento de preentrenamiento con el contexto recuperado en detalles específicos (nombres comerciales de fármacos).

Recomendaciones de mejora. Aumentar top_n de 5 a 10 fragmentos recuperados podría reducir los fallos en enfermedades con síntomas genéricos. La aplicación de query expansion —añadiendo sinónimos clínicos a las consultas en lenguaje coloquial— también mejoraría el componente BM25 de la recuperación. A nivel del módulo de generación, un modelo más grande (≥ 3 B parámetros) aumentaría la fidelidad al contexto y reduciría las alucinaciones puntuales.

5. Experimentos adicionales

Para obtener el punto opcional de la práctica se realizaron dos experimentos complementarios: (T4.1) comparación del modelo generativo base con un modelo de mayor tamaño, y (T4.2) comparación de tres tamaños de contexto por fragmento recuperado. En ambos casos la configuración de recuperación fue idéntica (hybrid, $\alpha = 0,5$, top-N = 5) y se reutilizaron los embeddings cacheados, de forma que los experimentos sólo difirieron en el componente evaluado.

5.1 Comparación de modelos generativos (T4.1)

Se comparó el modelo base Llama-3.2-1B-Instruct-GGUF (1,24 B parámetros, 807 MB en disco) con llama3.2:3b (3 B parámetros, 2,0 GB en disco). Las 10 preguntas de evaluación se lanzaron con configuración idéntica sobre ambos modelos. Los resultados se muestran en la Tabla 6.

Tabla 6. Puntuaciones por pregunta: modelo 1B vs modelo 3B.

#	Enfermedad	1B Total	3B Total
Q1	Common cold	3/3	3/3
Q2	Influenza	1,5/3	1,5/3
Q3	Asthma	3/3	3/3
Q4	Migraine	2/3	2,5/3
Q5	Gout	2/3	3/3

#	Enfermedad	1B Total	3B Total
Q6	Hypothyroidism	1/3	2,5/3
Q7	Pneumonia	3/3	3/3
Q8	Rheumatoid arthritis	2/3	2,5/3
Q9	Malaria	0/3	0,5/3
Q10	Tuberculosis	0,5/3	0,5/3
Total		17,5/30	22/30

El modelo 3B mejora **+4,5 puntos** (73,3% vs 58,3%). Las ganancias se concentran en: Q6 (+1,5) donde el 3B recupera la enfermedad desde conocimiento paramétrico al fallar la recuperación; Q5 (+1) al evitar la alucinación “allopurinol (Zyrtec)”; Q4 y Q8 (+0,5 c/u) por mayor coherencia al integrar fragmentos mixtos. Los fallos de Q9 y Q10 persisten en ambos modelos porque su causa es un fallo de recuperación (el artículo esperado no está en el top-5), que requiere mejorar el indexado.

5.2 Comparación de tamaños de contexto por fragmento (T4.2)

El parámetro `max_context_chars` controla cuántos caracteres de cada fragmento recuperado se incluyen en el prompt del LLM. Se compararon tres valores: 500 (~75 palabras), 1 500 (baseline, ~230 palabras) y 3 000 (~460 palabras). El ranking de recuperación es idéntico en los tres casos.

Tabla 7. Puntuaciones totales por configuración de contexto.

max-context-chars	P	C	V	Total
500	5,5	5,5	6,5	17,5/30
1 500 (baseline)	6,5	5,0	6,0	17,5/30
3 000	6,0	4,0	5,5	15,5/30

Los resultados revelan una tendencia contraintuitiva: **más contexto no implica mejores respuestas**. Los totales 17,5 / 17,5 / 15,5 muestran que el mayor contexto empeora el resultado. Cuando el ranking es correcto, el contexto adicional es redundante. Cuando el ranking contiene ruido, los 3 000 chars de fragmentos incorrectos dominan el prompt: en Q8, los fragmentos de JIA en las primeras posiciones con 3 000 chars eclipsan el artículo de RA en posición 5; en Q6, el fragmento de Hyperkalemia lleva al modelo a diagnosticar “Hypokalemia” de forma errónea. En Q4 (Migraine), 500 chars funciona mejor porque trunca los fragmentos de Arnold–Chiari antes de que el modelo los procese. La conclusión es que el baseline de 1 500 chars ofrece el mejor equilibrio con el ranking actual.

6. Conclusiones

Este trabajo ha presentado RAGMED, un sistema de recuperación aumentada por generación orientado al dominio médico que, dado un conjunto de síntomas descritos en lenguaje natural, devuelve un diagnóstico diferencial fundamentado en un corpus de artículos de Wikipedia sobre enfermedades.

Sobre el módulo de adquisición de datos. El uso de la API REST de Wikipedia con el parámetro `explain_text` simplificó considerablemente el pipeline de extracción: el servidor devuelve el texto limpio con los encabezados de sección ya marcados, eliminando la necesidad de parsear HTML complejo. La estrategia de segmentación por sección —hasta cuatro fragmentos por enfermedad (Lead, Signs and symptoms, Causes, Treatment)— resultó más útil que una ventana de tokens fija, porque preserva la cohesión semántica de cada fragmento. El corpus final contiene 11 012 fragmentos procedentes de aproximadamente 5 390 enfermedades.

Sobre el módulo de recuperación. La función de recuperación híbrida (BM25 + similitud coseno, $\alpha = 0,5$) demostró ser superior a la recuperación puramente semántica para consultas en lenguaje natural. El componente BM25 refuerza la coincidencia de términos médicos exactos cuando el vocabulario del paciente coincide con el del corpus, mientras que el componente semántico captura la coherencia conceptual cuando la coincidencia léxica es parcial. Los resultados de evaluación confirman puntuaciones perfectas en las tres preguntas con mayor coincidencia terminológica (Q1, Q3, Q7).

Sobre la evaluación. El sistema base (modelo 1B, contexto de 1 500 caracteres) obtuvo **17,5/30 puntos (58,3 %)** en las diez preguntas. Esta puntuación refleja el comportamiento bimodal esperado en sistemas RAG sobre corpus de dominio general: rendimiento óptimo cuando la recuperación es correcta, y fallos cuando los síntomas son genéricos o el vocabulario de la consulta difiere del corpus. Los fallos principales se concentran en enfermedades tropicales y crónicas (malaria, tuberculosis, hipotiroidismo) y se deben al módulo de recuperación, no al generativo.

Sobre los experimentos adicionales. La sustitución del modelo de 1B por uno de 3B parámetros supuso la mejora más significativa: **+4,5 puntos (22/30, 73,3 %)**. El modelo más grande compensa parcialmente los fallos de recuperación gracias a su mayor conocimiento paramétrico, como se observó en Q6 (hipotiroidismo), donde identificó correctamente la enfermedad a pesar de que ningún fragmento recuperado pertenecía a ese artículo. Por el contrario, aumentar el contexto por fragmento de 1 500 a 3 000 caracteres empeoró el resultado global (15,5/30), confirmando que cuando el ranking contiene ruido, más contexto amplifica el señal erróneo.

Limitaciones y trabajo futuro. Las principales limitaciones del sistema son: (1) la brecha léxica entre lenguaje coloquial y terminología clínica, que perjudica al componente BM25; (2) la saturación del espacio de embeddings en síntomas genéricos compartidos por muchas enfermedades; y (3) el comportamiento de autorestricción del modelo base ante preguntas de índole médica. Como mejoras futuras se propone implementar *query expansion* con sinónimos clínicos, aumentar `top_n` para reducir los fallos por síntomas compartidos, y evaluar modelos de mayor tamaño o instrucción específica para el dominio médico.

Referencias

- [1] W. B. Croft, D. Metzler, and T. Strohman, *Search Engines: Information Retrieval in Practice*, Pearson Education, 2015.
- [2] Wikimedia Foundation, “MediaWiki API,” Wikipedia. [Online]. Available: https://www.mediawiki.org/wiki/API:Main_page. [Accessed: May 2026].
- [3] Wikipedia contributors, “List of diseases,” Wikipedia, The Free Encyclopedia. [Online]. Available: [https://en.wikipedia.org/wiki/List_of_diseases_\(A\)](https://en.wikipedia.org/wiki/List_of_diseases_(A)). [Accessed: May 2026].
- [4] L. Richardson, “Beautiful Soup Documentation,” Crummy.com. [Online]. Available: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. [Accessed: May 2026].
- [5] Ollama, “Ollama — Get up and running with large language models.” [Online]. Available: <https://ollama.com/>. [Accessed: May 2026].
- [6] D. Brown, “rank-bm25: A two-line search engine,” GitHub. [Online]. Available: https://github.com/dorianbrown/rank_bm25. [Accessed: May 2026].